

ASSIGNMENT – 2

Regression Model Evaluation and Performance Analysis

AIM:

To implement a regression model for predicting a continuous target variable, evaluate its performance using multiple evaluation metrics, visualize prediction results, and analyze overfitting, underfitting, and bias-variance. The dataset used is a real-world loan-amount prediction dataset (Kaggle: Predict Loan Amount Data) containing numerical and categorical features related to loan applications, with the target variable being the loan amount sanctioned. Linear Regression is used as the baseline model, and Ridge, Lasso, and Elastic Net regression are used as regularized models, tuned using GridSearchCV, to control model complexity and reduce overfitting. Libraries used: pandas, numpy, matplotlib, scikit-learn.

SOURCE CODE:

```
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import cross_validate

from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("./train.csv")

print(df.head())
print(df.info())
print(df.describe())

# 3a. Handle missing values
df = df.ffill().bfill()
print("Missing values after imputation:\n", df.isnull().sum())

# 3b. Encode categorical variables
encoder = LabelEncoder()
for column in df.columns:
    if df[column].dtype == "object" or pd.api.types.is_string_dtype(df[column]):
        df[column] = encoder.fit_transform(df[column])

TARGET = "Loan Sanction Amount (USD)"
```

```
X = df.drop(TARGET, axis=1)
y = df[TARGET]
feature_names = X.columns

# 3c. Standardize numerical features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# 4. Train / Validation / Test Split
X_train, X_temp, y_train, y_temp = train_test_split(
X, y, test_size=0.30, random_state=42
)
X_validation, X_test, y_validation, y_test = train_test_split(
X_temp, y_temp, test_size=0.50, random_state=42
)
print("Training Samples:", len(X_train))
print("Validation Samples:", len(X_validation))
print("Testing Samples :", len(X_test))

# 5. Exploratory Data Analysis
print("summary staticis")
# print(df.describe())

# Visualize feature distributions and target distribution -----

#using the histogram
plt.figure(figsize=(6, 4))
plt.hist(y, bins=20)
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()

# Feature distribution plots
plt.figure(figsize=(15, 10))
grid_rows = int(np.ceil(len(feature_names) / 4))
for i, column in enumerate(feature_names, 1):
plt.subplot(grid_rows, 4, i) # 3 rows, 4 columns
plt.hist(df[column], bins=20)
plt.title(column)
plt.xlabel(column)
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# Feature vs Target scatter plots

plt.figure(figsize=(15, 10))
for i, column in enumerate(feature_names, 1):
```

```
plt.subplot(grid_rows, 4, i)
plt.scatter(df[column], y)
plt.title(column)
plt.xlabel(column)
plt.ylabel("Loan Amount")

plt.tight_layout()
plt.show()

# Split the dataset into training, validation and testing sets -----

X_train, X_temp, y_train, y_temp = train_test_split(
X, y, test_size=0.30, random_state=42
)

X_validation, X_test, y_validation, y_test = train_test_split(
X_temp, y_temp, test_size=0.50, random_state=42
)

# Helper: compute MAE, MSE, RMSE, R2 in one call
#y_true contains the actual values.
# y_pred contains the predicted values from the model.
def calc_metrics(y_true, y_pred):
mae = mean_absolute_error(y_true, y_pred) # actual - predicated = error take average of this errors
mse = mean_squared_error(y_true, y_pred) # saquare the errors usefull in the big value
rmse = np.sqrt(mse) # root of the mse
r2 = r2_score(y_true, y_pred) # tell the variation of the model preidction and the actual value
return mae, mse, rmse, r2

# Linear Regression tries to use all features.
# Ridge Regression also keeps all features but reduces their influence.
# Lasso Regression Hair Color and Favorite Food don't help predict the loan amount. I'll remove them.
# Elastic Regression Combines both advantages of the Ridge and Lasso:
# Ridge → Keeps all features but reduces their importance.
# Lasso → Can completely remove unimportant features.
# Elastic Net → Does bot

from sklearn.linear_model import LinearRegression
# 6. Baseline Linear Regression
print("----- Linear Regression -----")
# equation = y=mx+c
#used whenever the op is number
#dataset is regression

linear = LinearRegression()
start = time.time()
linear.fit(X_train, y_train)
```

```
linear_time = time.time() - start

# Predict using validation and test data
val_pred_linear = linear.predict(X_validation)
test_pred_linear = linear.predict(X_test)

# Calculate validation metrics
lin_val_mae, lin_val_mse, lin_val_rmse, lin_val_r2 = calc_metrics(y_validation, val_pred_linear)

# Calculate test metrics
lin_mae, lin_mse, lin_rmse, lin_r2 = calc_metrics(y_test, test_pred_linear)

print("Validation -> MAE:", lin_val_mae, "MSE:", lin_val_mse, "RMSE:", lin_val_rmse, "R2:", lin_val_r2)
print("Test -> MAE:", lin_mae, "MSE:", lin_mse, "RMSE:", lin_rmse, "R2:", lin_r2)
print("Training Time :", linear_time)

# 7. Ridge Regression
#adds penalty if the model try to learn by memorizing
# alpha is the penatly here and we don't know which alpha is best so we try some bunch of alpha value

ridge_model = Ridge()

parameters = {"alpha": [0.01, 0.1, 1, 10, 100]} #creating dict containig all the alpha values

grid_ridge = GridSearchCV(ridge_model, parameters, cv=5) # ridge model = model to train on, paramentes
= the value to test,cv=5 the method to use
start = time.time()
grid_ridge.fit(X_train, y_train)
ridge_time = time.time() - start

print("Best Parameters:", grid_ridge.best_params_)
print("Best Cross Validation R2:", grid_ridge.best_score_)

val_pred_ridge = grid_ridge.predict(X_validation)
test_pred_ridge = grid_ridge.predict(X_test)

ridge_val_mae, ridge_val_mse, ridge_val_rmse, ridge_val_r2 = calc_metrics(y_validation, val_pred_ridge)
ridge_mae, ridge_mse, ridge_rmse, ridge_r2 = calc_metrics(y_test, test_pred_ridge)

print("Validation -> MAE:", ridge_val_mae, "MSE:", ridge_val_mse, "RMSE:", ridge_val_rmse, "R2:",
ridge_val_r2)
print("Test -> MAE:", ridge_mae, "MSE:", ridge_mse, "RMSE:", ridge_rmse, "R2:", ridge_r2)

# 8. Lasso Regression
print("----- Lasso Regression -----")
#use the regularized model and then just like the ridgemodel
#It can completely remove unimportant features from the model.
#good when the fetures are unesscary
```

```
lasso = Lasso(max_iter=5000) #no of iterations
lasso_params = {"alpha": [0.001, 0.01, 0.1, 1, 10]} #different parameter to work on

grid_lasso = GridSearchCV(lasso, lasso_params, cv=5)
start = time.time()
grid_lasso.fit(X_train, y_train)
lasso_time = time.time() - start

print("Best Parameters :", grid_lasso.best_params_)
print("Best CV R2 :", grid_lasso.best_score_)

val_pred_lasso = grid_lasso.predict(X_validation)
test_pred_lasso = grid_lasso.predict(X_test)

lasso_val_mae, lasso_val_mse, lasso_val_rmse, lasso_val_r2 = calc_metrics(y_validation, val_pred_lasso)
lasso_mae, lasso_mse, lasso_rmse, lasso_r2 = calc_metrics(y_test, test_pred_lasso)

print("Validation -> MAE:", lasso_val_mae, "MSE:", lasso_val_mse, "RMSE:", lasso_val_rmse, "R2:",
lasso_val_r2)
print("Test -> MAE:", lasso_mae, "MSE:", lasso_mse, "RMSE:", lasso_rmse, "R2:", lasso_r2)


# 9. Elastic Net Regression
#combines both the lasso and then ridge and then remove unesscary fetures and then also minmize there
importance
print("----- Elastic Net Regression -----")

elastic = ElasticNet(max_iter=5000)
elastic_params = {
    "alpha": [0.01, 0.1, 1, 10],
    "l1_ratio": [0.2, 0.5, 0.8] # how much the ridge and the lasso are mixed why three value because we
don't which mix is best that's why we give three value
}

grid_elastic = GridSearchCV(elastic, elastic_params, cv=5)
start = time.time()
grid_elastic.fit(X_train, y_train)
elastic_time = time.time() - start

print("Best Parameters :", grid_elastic.best_params_)
print("Best CV R2 :", grid_elastic.best_score_)

val_pred_elastic = grid_elastic.predict(X_validation)
test_pred_elastic = grid_elastic.predict(X_test)

elastic_val_mae, elastic_val_mse, elastic_val_rmse, elastic_val_r2 = calc_metrics(y_validation,
val_pred_elastic)
elastic_mae, elastic_mse, elastic_rmse, elastic_r2 = calc_metrics(y_test, test_pred_elastic)
```

```
print("Validation -> MAE:", elastic_val_mae, "MSE:", elastic_val_mse, "RMSE:", elastic_val_rmse, "R2:",  
      elastic_val_r2)  
print("Test -> MAE:", elastic_mae, "MSE:", elastic_mse, "RMSE:", elastic_rmse, "R2:", elastic_r2)
```

#hypermeter is the value we set before the training such as alpha the model doesn't know we give it
#5 folds or 5cv is the process of splitting the dataset into the five sets and using the four sets for
the training and then the last set for the

10. Table 1 - Hyperparameter Tuning Summary

```
table1 = pd.DataFrame({  
    "Model": ["Ridge Regression", "Lasso Regression", "Elastic Net Regression"],  
    "Search Method": ["Grid Search", "Grid Search", "Grid Search"],  
    "Best Parameters": [grid_ridge.best_params_, grid_lasso.best_params_, grid_elastic.best_params_],  
    "Best Cross Validation R2": [grid_ridge.best_score_, grid_lasso.best_score_, grid_elastic.best_score_]  
})  
print("\nTable 1: Hyperparameter Tuning Summary")  
print(table1)
```

11. Table 2 - Validation Performance Summary

```
table2 = pd.DataFrame({  
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "Elastic Net"],  
    "MAE": [lin_val_mae, ridge_val_mae, lasso_val_mae, elastic_val_mae],  
    "MSE": [lin_val_mse, ridge_val_mse, lasso_val_mse, elastic_val_mse],  
    "RMSE": [lin_val_rmse, ridge_val_rmse, lasso_val_rmse, elastic_val_rmse],  
    "R2 Score": [lin_val_r2, ridge_val_r2, lasso_val_r2, elastic_val_r2]  
})  
print("\nTable 2: Validation Performance Summary")  
print(table2)
```

12. Table 3 - Test Set Performance

```
table3 = pd.DataFrame({  
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "Elastic Net"],  
    "MAE": [lin_mae, ridge_mae, lasso_mae, elastic_mae],  
    "MSE": [lin_mse, ridge_mse, lasso_mse, elastic_mse],  
    "RMSE": [lin_rmse, ridge_rmse, lasso_rmse, elastic_rmse],  
    "R2 Score": [lin_r2, ridge_r2, lasso_r2, elastic_r2],  
    "Training Time (s)": [linear_time, ridge_time, lasso_time, elastic_time]  
})  
print("\nTable 3: Test Set Performance")  
print(table3)
```

13. Table 4 - Coefficient Comparison (per feature)

```
# -----
table4 = pd.DataFrame({
    "Feature": feature_names,
    "Linear": linear.coef_,
    "Ridge": grid_ridge.best_estimator_.coef_,
    "Lasso": grid_lasso.best_estimator_.coef_,
    "Elastic Net": grid_elastic.best_estimator_.coef_
})
print("\nTable 4: Coefficient Comparison")
print(table4)

# 14. Visualizations - Predicted vs Actual & Residuals (all models)

#sns is built on over the matplotlib

model_preds = { #stores model name and its predications
    "Linear Regression": test_pred_linear,
    "Ridge Regression": test_pred_ridge,
    "Lasso Regression": test_pred_lasso,
    "Elastic Net": test_pred_elastic
}

for name, preds in model_preds.items():
    # Predicted vs Actual
    plt.figure(figsize=(6, 6))
    plt.scatter(y_test, preds)
    plt.xlabel("Actual Values")
    plt.ylabel("Predicted Values")
    plt.title(f"Actual vs Predicted - {name}")
    plt.show()

    # Residual plot
    #residual = actual - predicated
    # here y-test is actual and the pred is predicted
    residual = y_test - preds
    plt.figure(figsize=(6, 4))
    plt.scatter(preds, residual)
    plt.axhline(y=0)
    plt.xlabel("Predicted Values")
    plt.ylabel("Residual")
    plt.title(f"Residual Plot - {name}")
    plt.show()

# 15. Training vs Validation Error across Ridge alphas
#why we do this is because to find the overfitting and the underfitting

#created two empty list to store train and the validation error
train_error = []
validation_error = []
```

```
alphas = [0.01, 0.1, 1, 10, 100] #range of the alphas

for alpha in alphas:
    model = Ridge(alpha=alpha)
    model.fit(X_train, y_train)
    train_prediction = model.predict(X_train)
    validation_prediction = model.predict(X_validation)

#here the mean_squared_error is a function form the sckit learn
train_error.append(mean_squared_error(y_train, train_prediction))
validation_error.append(mean_squared_error(y_validation, validation_prediction))

plt.figure(figsize=(6, 4))
plt.plot(alphas, train_error, marker="o", label="Training Error")
plt.plot(alphas, validation_error, marker="o", label="Validation Error")
plt.xscale("log")
plt.xlabel("Alpha")
plt.ylabel("MSE")
plt.title("Training vs Validation Error (Ridge)")
plt.legend()
plt.show()

#

# 16. Coefficient Comparison Bar Plot (average magnitude)

# Suppose your model is

# Loan Amount =
# 5000
# + 120 × Income
# + 80 × Credit Score
# - 30 × Age
# Then the coefficients are 120,80,30

# Linear Regression → Uses the original coefficients.
# Ridge Regression → Shrinks coefficients slightly.
# Lasso Regression → Shrinks some coefficients to zero (feature selection).
# Elastic Net → Combination of Ridge and Lasso.

#this graph is used to know how each model is compressed coefficients

models_list = ["Linear", "Ridge", "Lasso", "ElasticNet"]
coefficients = [
    #np.abs = abosulate value ie negative values become postive
    #best_estimator_ returns the best model
    #.coef = to get its coefficients
    np.mean(np.abs(linear.coef_)),
    np.mean(np.abs(grid_ridge.best_estimator_.coef_)),
    np.mean(np.abs(grid_lasso.best_estimator_.coef_)),
    np.mean(np.abs(grid_elastic.best_estimator_.coef_))
]
```



```
plt.figure(figsize=(6, 4))
plt.bar(models_list, coefficients)
plt.title("Coefficient Comparison (Average Magnitude)")
plt.ylabel("Average Coefficient Magnitude")
plt.show()

# 17. Table 5 - Training vs Validation Error (for Overfitting/Underfitting analysis)
train_pred_linear = linear.predict(X_train)
train_pred_ridge = grid_ridge.predict(X_train)
train_pred_lasso = grid_lasso.predict(X_train)
train_pred_elastic = grid_elastic.predict(X_train)

train_mse_linear = mean_squared_error(y_train, train_pred_linear)
train_mse_ridge = mean_squared_error(y_train, train_pred_ridge)
train_mse_lasso = mean_squared_error(y_train, train_pred_lasso)
train_mse_elastic = mean_squared_error(y_train, train_pred_elastic)

table5 = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "Elastic Net"],
    "Training MSE": [train_mse_linear, train_mse_ridge, train_mse_lasso, train_mse_elastic],
    "Validation MSE": [lin_val_mse, ridge_val_mse, lasso_val_mse, elastic_val_mse],
    "Gap (Val - Train)": [
        lin_val_mse - train_mse_linear,
        ridge_val_mse - train_mse_ridge,
        lasso_val_mse - train_mse_lasso,
        elastic_val_mse - train_mse_elastic
    ]
})
print("\nTable 5: Training vs Validation MSE (use this for the Overfitting/Underfitting write-up)")
print(table5)

print("\nAll tables generated: table1, table2, table3, table4, table5")
```

OUTPUT:

Customer ID Name ... Property Price Loan Sanction Amount (USD)

```
0 C-36995 Frederica Shealy ... 119933.46 54607.18
1 C-33999 America Calderone ... 54791.00 37469.98
2 C-3770 Rosetta Verne ... 72440.58 36474.43
3 C-26480 Zoe Chitty ... 121441.51 56040.54
4 C-23459 Afton Venema ... 208567.91 74008.28
```

```
[5 rows x 24 columns]
<class 'pandas.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 24 columns):
# Column Non-Null Count Dtype
---
0 Customer ID 30000 non-null str
1 Name 30000 non-null str
2 Gender 29947 non-null str
3 Age 30000 non-null int64
4 Income (USD) 25424 non-null float64
5 Income Stability 28317 non-null str
```

```
6 Profession 30000 non-null str
7 Type of Employment 22730 non-null str
8 Location 30000 non-null str
9 Loan Amount Request (USD) 30000 non-null float64
10 Current Loan Expenses (USD) 29828 non-null float64
11 Expense Type 1 30000 non-null str
12 Expense Type 2 30000 non-null str
13 Dependents 27507 non-null float64
14 Credit Score 28297 non-null float64
15 No. of Defaults 30000 non-null int64
16 Has Active Credit Card 28434 non-null str
17 Property ID 30000 non-null int64
18 Property Age 25150 non-null float64
19 Property Type 30000 non-null int64
20 Property Location 29644 non-null str
21 Co-Applicant 30000 non-null int64
22 Property Price 30000 non-null float64
23 Loan Sanction Amount (USD) 29660 non-null float64
dtypes: float64(8), int64(5), str(11)
memory usage: 5.5 MB
None
Age Income (USD) ... Property Price Loan Sanction Amount (USD)
count 30000.000000 2.542400e+04 ... 3.000000e+04 29660.000000
mean 40.092300 2.630574e+03 ... 1.317597e+05 47649.342208
std 16.045129 1.126272e+04 ... 9.354955e+04 48221.146686
min 18.000000 3.777000e+02 ... -9.990000e+02 -999.000000
25% 25.000000 1.650457e+03 ... 6.057216e+04 0.000000
50% 40.000000 2.222435e+03 ... 1.099936e+05 35209.395000
75% 55.000000 3.090593e+03 ... 1.788807e+05 74261.250000
max 65.000000 1.777460e+06 ... 1.077967e+06 481907.320000

[8 rows x 13 columns]
Missing values after imputation:
Customer ID 0
Name 0
Gender 0
Age 0
Income (USD) 0
Income Stability 0
Profession 0
Type of Employment 0
Location 0
Loan Amount Request (USD) 0
Current Loan Expenses (USD) 0
Expense Type 1 0
Expense Type 2 0
Dependents 0
Credit Score 0
No. of Defaults 0
Has Active Credit Card 0
Property ID 0
Property Age 0
Property Type 0
Property Location 0
Co-Applicant 0
Property Price 0
Loan Sanction Amount (USD) 0
dtype: int64
Training Samples: 21000
Validation Samples: 4500
Testing Samples : 4500
summary staticis
```

----- Linear Regression -----

Validation -> MAE: 20950.571275447004 MSE: 894121981.3093051 RMSE: 29901.87253851011 R2: 0.6098185499573614

Test -> MAE: 21034.65288715323 MSE: 928038905.4810158 RMSE: 30463.730984254307 R2: 0.5917735561996129

Training Time : 0.01958632469177246

Best Parameters: {'alpha': 0.1}

Best Cross Validation R2: 0.6006850248938804

Validation -> MAE: 20950.22566608999 MSE: 894111818.1110933 RMSE: 29901.702595522773 R2: 0.6098229850249453

Test -> MAE: 21034.489081279815 MSE: 928024105.3834791 RMSE: 30463.488069876028 R2: 0.5917800664775221

----- Lasso Regression -----

Best Parameters : {'alpha': 10}

Best CV R2 : 0.600691388883308

Validation -> MAE: 20936.46776472812 MSE: 893844465.4258325 RMSE: 29897.23173515957 R2: 0.6099396537352426

Test -> MAE: 21021.902985370776 MSE: 921582183.3447788 RMSE: 30357.57209239202 R2: 0.5946137439338944

----- Elastic Net Regression -----

Best Parameters : {'alpha': 0.01, 'l1_ratio': 0.8}

Best CV R2 : 0.6006487478692256

Validation -> MAE: 20945.54736206528 MSE: 893859742.3630247 RMSE: 29897.487224899432 R2: 0.6099329871085065

Test -> MAE: 21035.541357617098 MSE: 925190420.4209872 RMSE: 30416.94298283421 R2: 0.5930265499258525

Table 1: Hyperparameter Tuning Summary

Model ... Best Cross Validation R2

0 Ridge Regression ... 0.600685

1 Lasso Regression ... 0.600691

2 Elastic Net Regression ... 0.600649

[3 rows x 4 columns]

Table 2: Validation Performance Summary

Model MAE MSE RMSE R2 Score

0 Linear Regression 20950.571275 8.941220e+08 29901.872539 0.609819

1 Ridge Regression 20950.225666 8.941118e+08 29901.702596 0.609823

2 Lasso Regression 20936.467765 8.938445e+08 29897.231735 0.609940

3 Elastic Net 20945.547362 8.938597e+08 29897.487225 0.609933

Table 3: Test Set Performance

Model MAE ... R2 Score Training Time (s)

0 Linear Regression 21034.652887 ... 0.591774 0.019586

1 Ridge Regression 21034.489081 ... 0.591780 0.195013

2 Lasso Regression 21021.902985 ... 0.594614 57.712157

3 Elastic Net 21035.541358 ... 0.593027 13.270735

[4 rows x 6 columns]

Table 4: Coefficient Comparison

Feature Linear ... Lasso Elastic Net

0 Customer ID -8741.044484 ... -8732.419563 -8725.674356

1 Name -190.499039 ... -180.723053 -192.383577

2 Gender -87.284676 ... -75.868963 -85.200692

3 Age -579.844429 ... -566.322590 -572.126731

4 Income (USD) 19839.798114 ... -0.000000 1093.101980

5 Income Stability 255.167671 ... 240.658820 252.679462

6 Profession -34.075478 ... -19.025405 -34.618963

7 Type of Employment 98.152941 ... 91.175538 98.496788

8 Location 75.651880 ... 71.529957 83.994947

9 Loan Amount Request (USD) 34034.584434 ... 33831.209473 33326.372021

10 Current Loan Expenses (USD) -488.481750 ... -456.552525 -403.736031

11 Expense Type 1 21.799902 ... 7.676743 22.683669

12 Expense Type 2 405.494824 ... 395.322279 401.857041

13 Dependents -67.377237 ... -54.013800 -65.107249

14 Credit Score 11311.683942 ... 11298.984238 11297.350949

15 No. of Defaults -351.253730 ... -343.515613 -352.963745

16 Has Active Credit Card -131.788979 ... -119.664439 -130.857088

```
17 Property ID -205.050654 ... -198.560396 -205.889519
18 Property Age -21619.717011 ... -1498.818767 -2752.283476
19 Property Type 259.317707 ... 249.403937 259.312590
20 Property Location 308.186285 ... 299.555138 310.265302
21 Co-Applicant -53.098392 ... -44.723356 -57.874081
22 Property Price -1022.690947 ... -835.500026 -395.304669
```

[23 rows x 5 columns]

Table 5: Training vs Validation MSE (use this for the Overfitting/Underfitting write-up)

Model Training MSE Validation MSE Gap (Val - Train)

0 Linear Regression 9.304187e+08 8.941220e+08 -3.629674e+07

1 Ridge Regression 9.304188e+08 8.941118e+08 -3.630699e+07

2 Lasso Regression 9.305897e+08 8.938445e+08 -3.674525e+07

3 Elastic Net 9.306122e+08 8.938597e+08 -3.675250e+07

All tables generated: table1, table2, table3, table4, table5

VISUALIZATIONS:

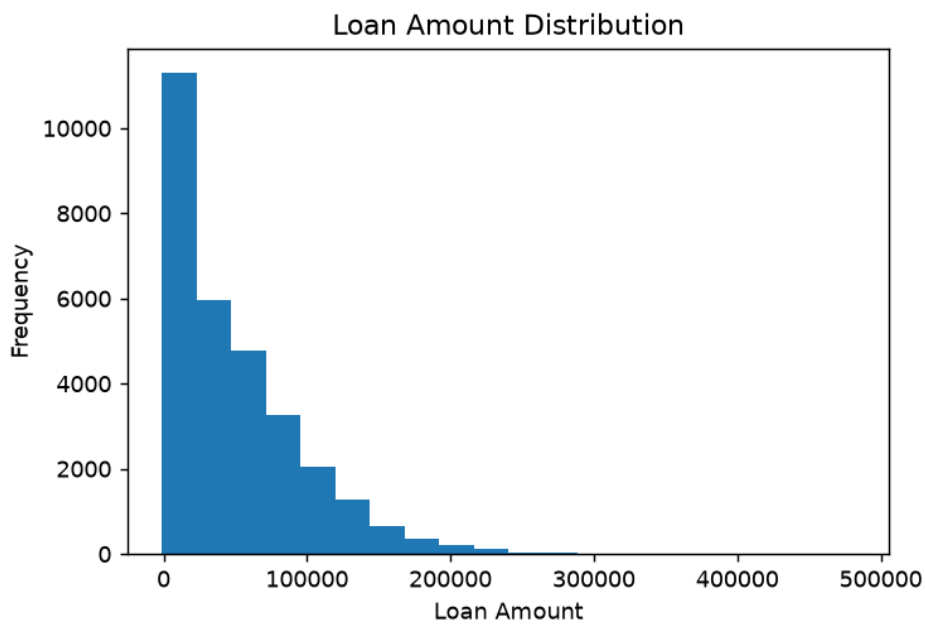


Fig 1. Loan Amount Distribution (target variable)

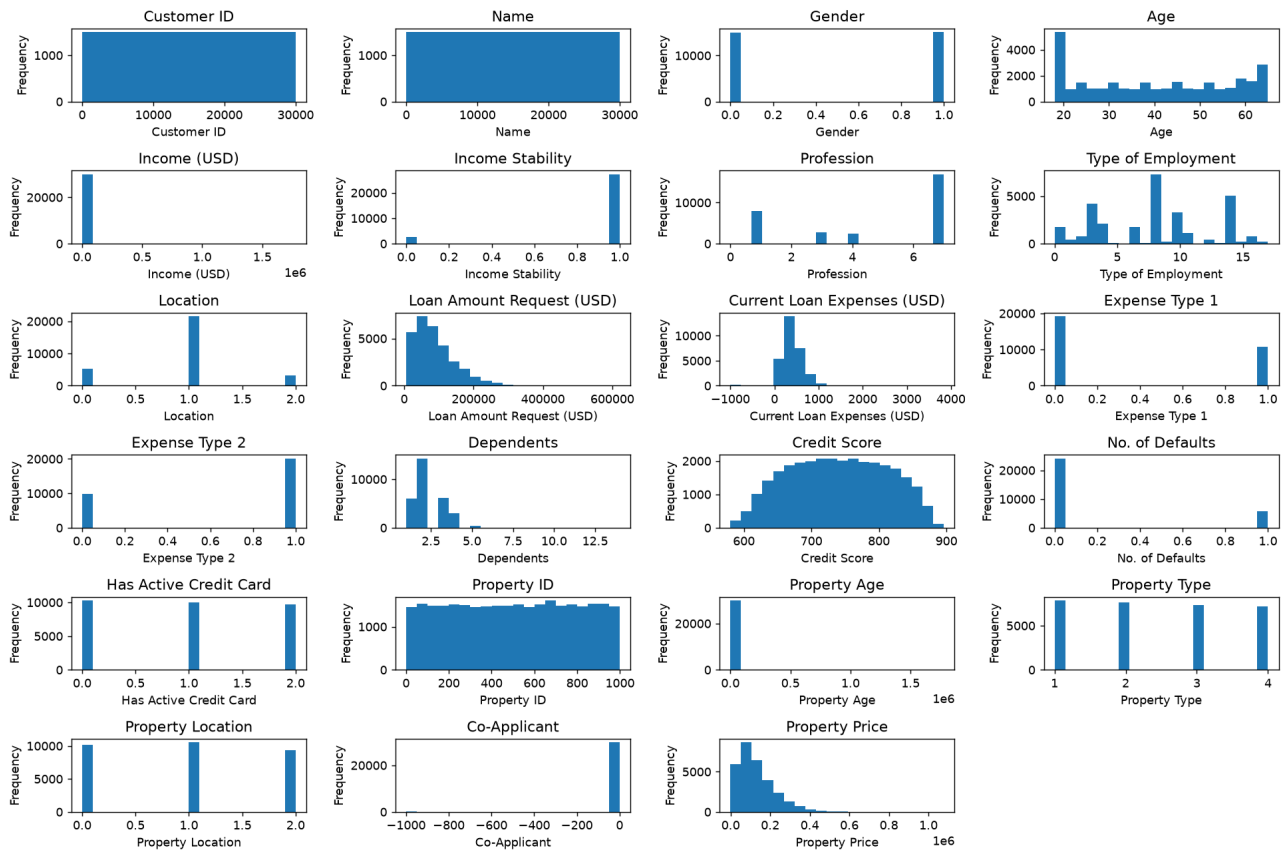


Fig 2. Feature distribution histograms (all 23 features)

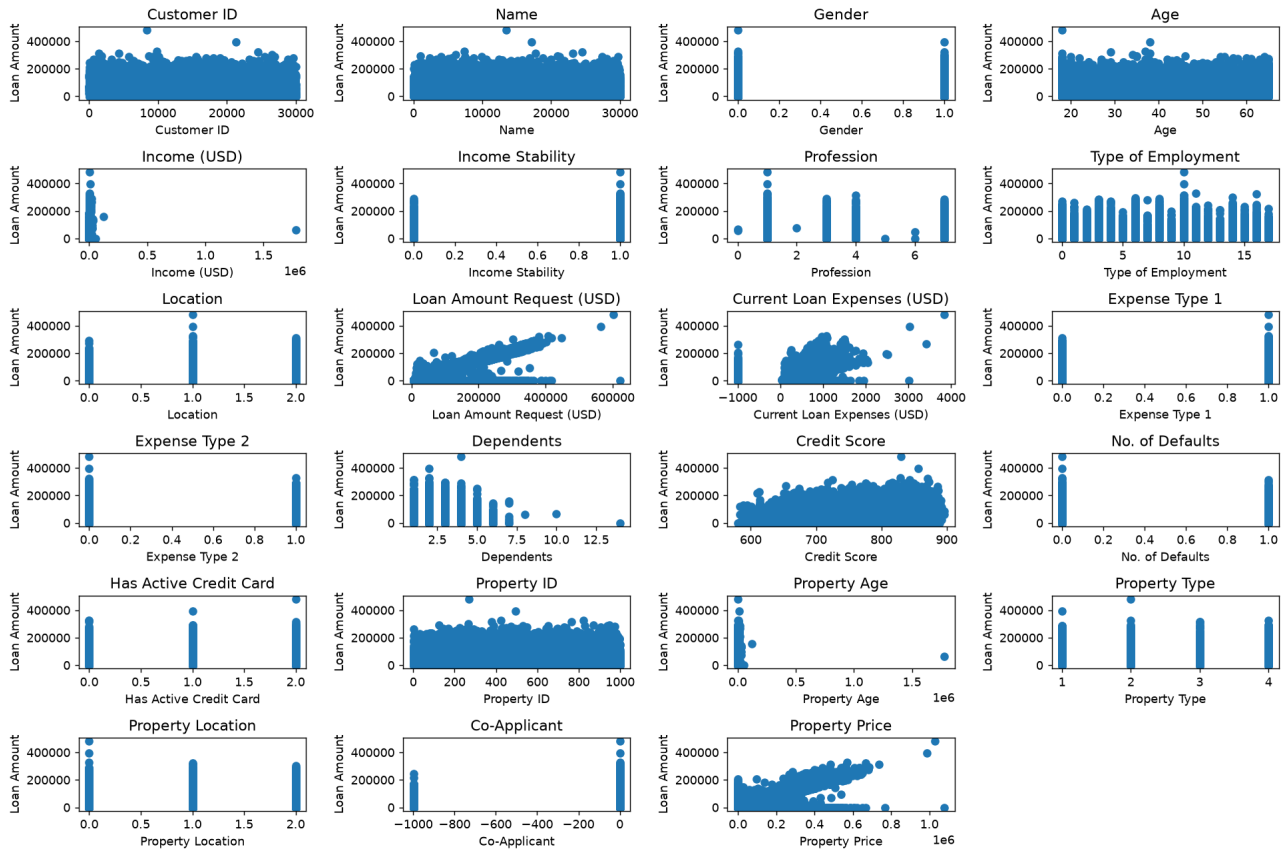


Fig 3. Feature vs Target scatter plots (all 23 features)

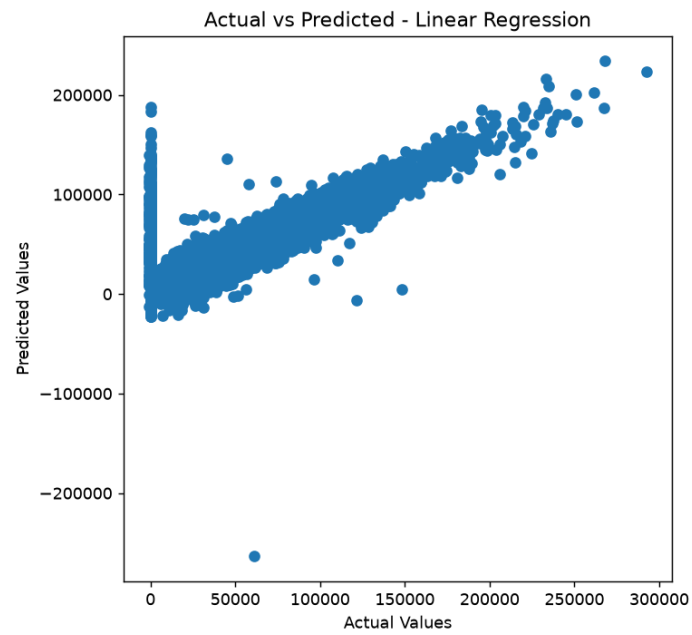


Fig 4. Actual vs Predicted - Linear Regression

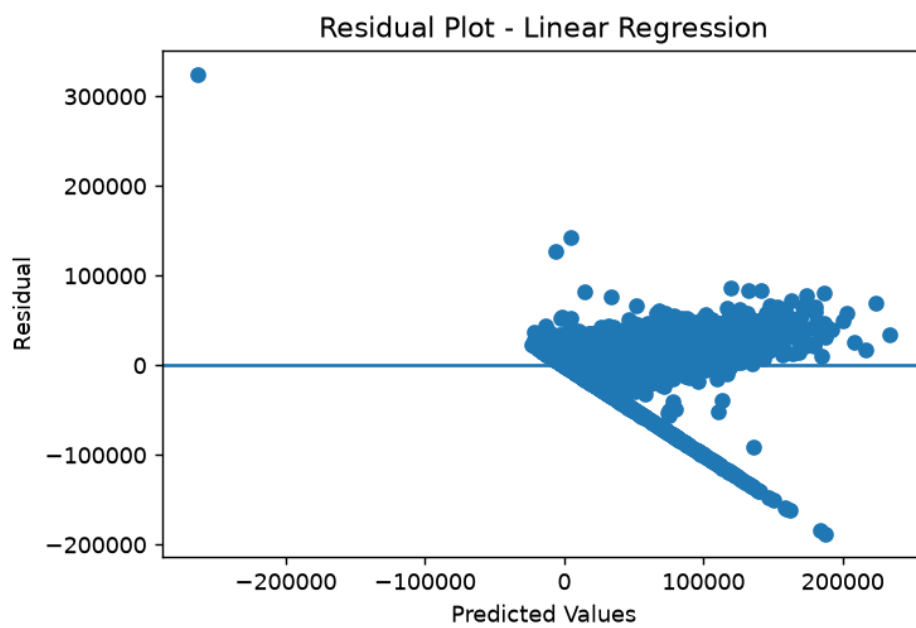


Fig 5. Residual Plot - Linear Regression

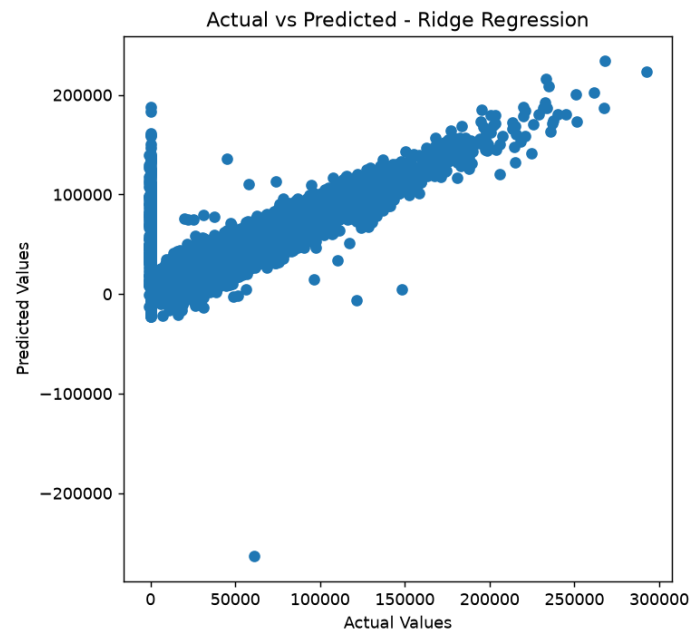


Fig 6. Actual vs Predicted - Ridge Regression

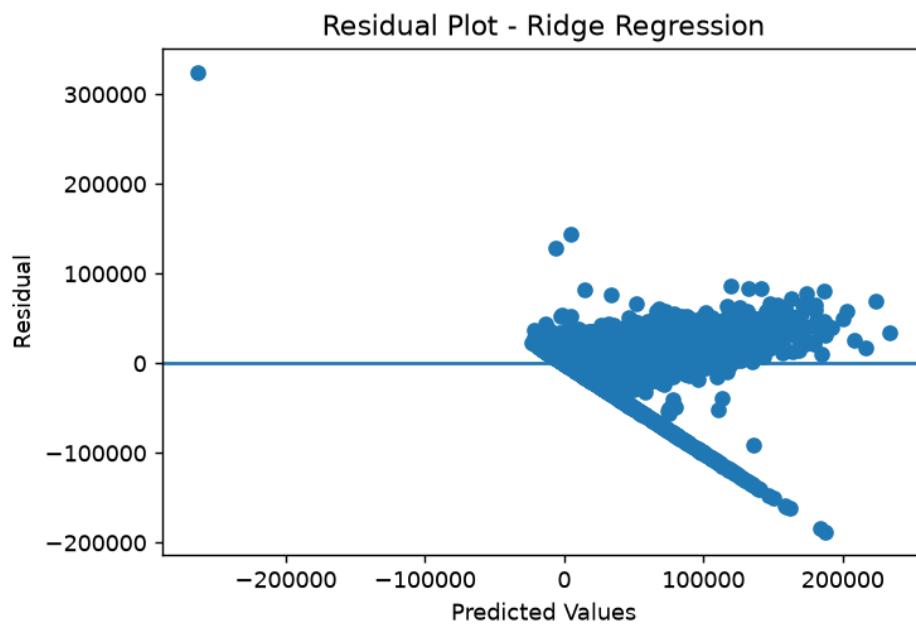


Fig 7. Residual Plot - Ridge Regression

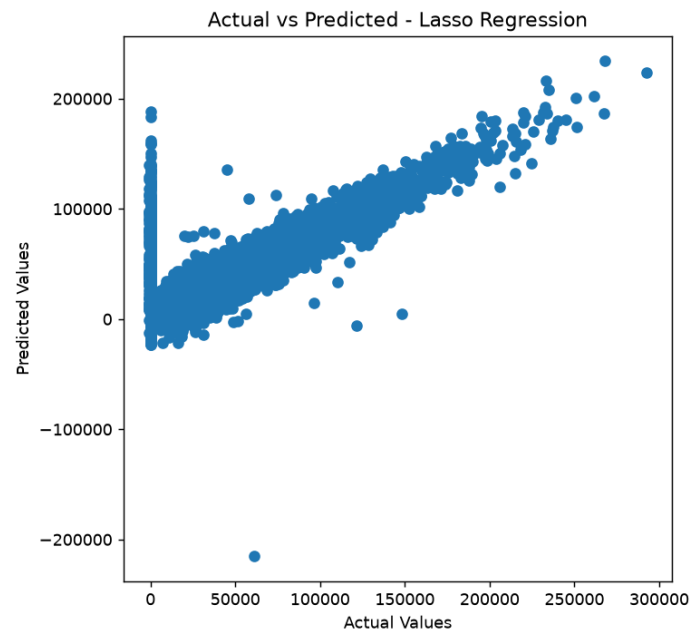


Fig 8. Actual vs Predicted - Lasso Regression

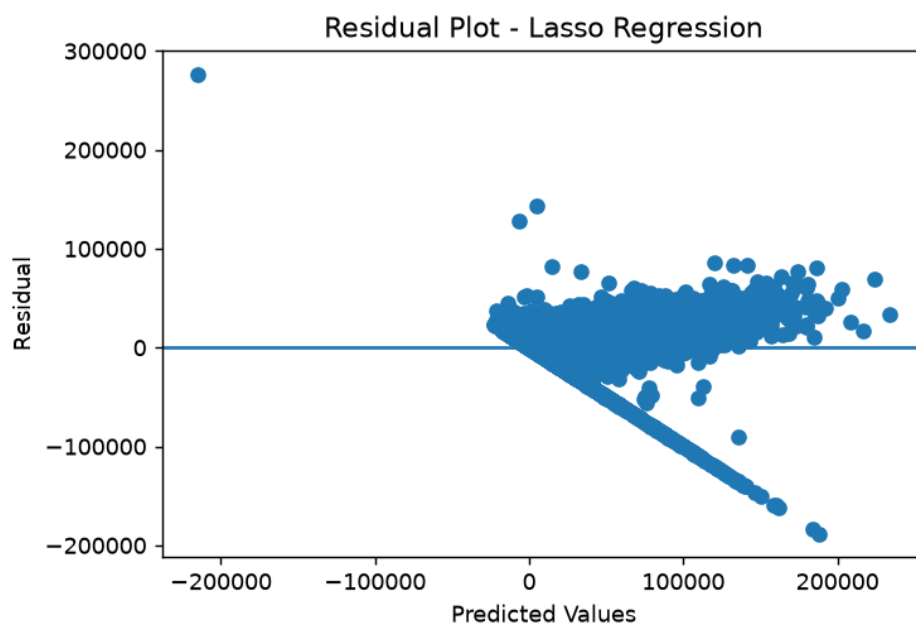


Fig 9. Residual Plot - Lasso Regression

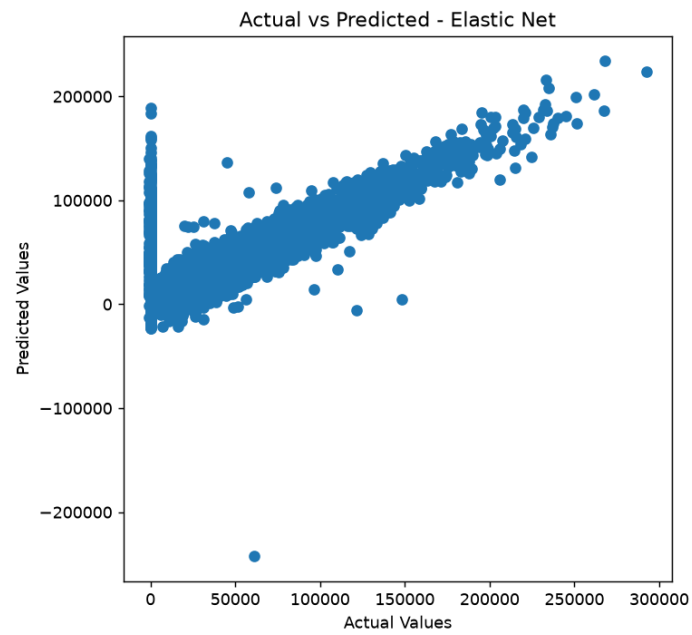


Fig 10. Actual vs Predicted - Elastic Net

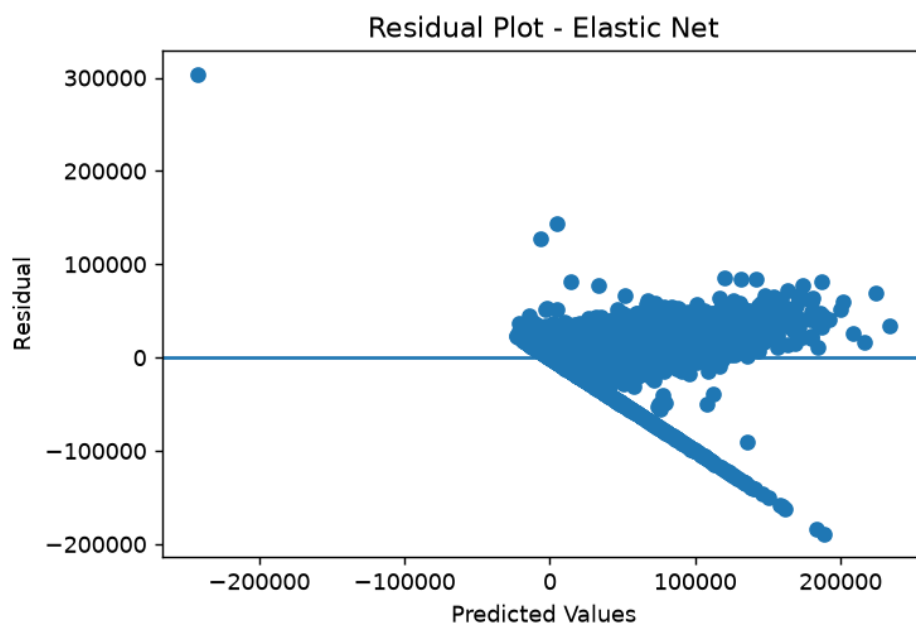


Fig 11. Residual Plot - Elastic Net

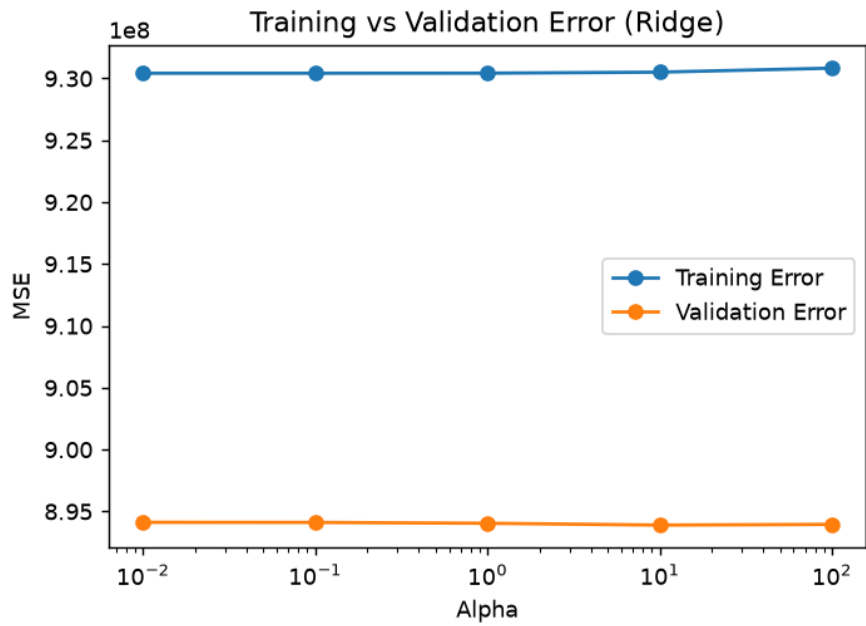


Fig 12. Training vs Validation Error across Ridge alphas

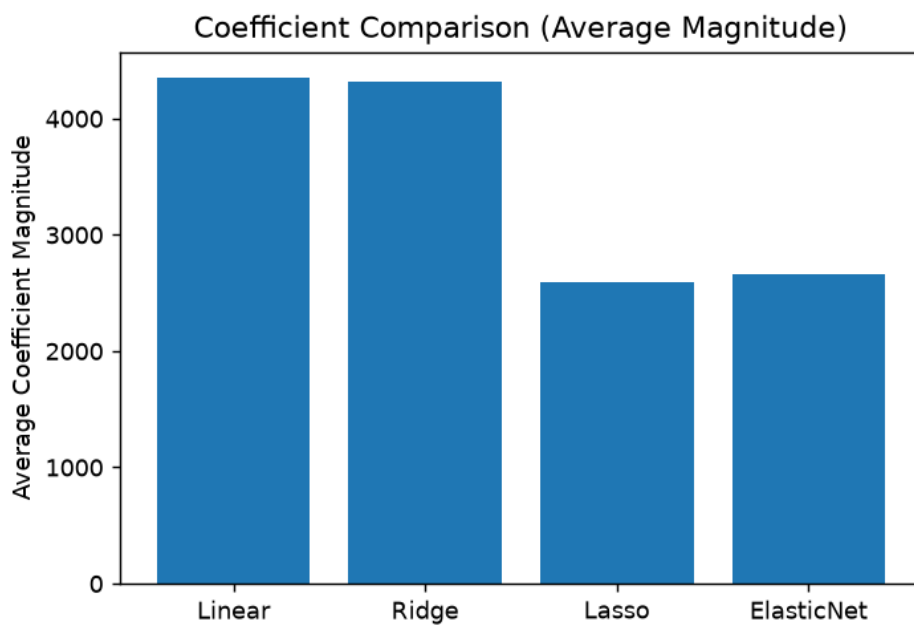


Fig 13. Coefficient Comparison (average magnitude) across models

RESULT TABLES:

Table 2: Validation Performance Summary

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	20950.57	8.9412e+08	29901.87	0.6098
Ridge Regression	20950.23	8.9411e+08	29901.70	0.6098
Lasso Regression	20936.47	8.9384e+08	29897.23	0.6099

Model	MAE	MSE	RMSE	R2 Score
Elastic Net	20945.55	8.9386e+08	29897.49	0.6099

Table 3: Test Set Performance

Model	MAE	MSE	RMSE	R2 Score	Training Time (s)
Linear Regression	21034.65	9.2804e+08	30463.73	0.5918	0.0196
Ridge Regression	21034.49	9.2802e+08	30463.49	0.5918	0.1950
Lasso Regression	21021.90	9.2158e+08	30357.57	0.5946	57.71
Elastic Net	21035.54	9.2519e+08	30416.94	0.5930	13.27

Table 5: Training vs Validation MSE (Overfitting / Underfitting)

Model	Training MSE	Validation MSE	Gap (Val - Train)
Linear Regression	9.3042e+08	8.9412e+08	-3.6297e+07
Ridge Regression	9.3042e+08	8.9411e+08	-3.6307e+07
Lasso Regression	9.3059e+08	8.9384e+08	-3.6745e+07
Elastic Net	9.3061e+08	8.9386e+08	-3.6753e+07

TASKS PERFORMED:

#	Task	Description
1	Loaded and inspected the dataset	30000 rows x 24 columns loaded from train.csv; checked info(), describe(), and null counts.
2	Handled missing values	Applied forward-fill then backward-fill (ffill/bfill); confirmed 0 nulls remaining across all columns.
3	Encoded categorical variables	Label-encoded every text/string column so all 24 columns became numeric.
4	Standardized numerical features	Scaled all 23 feature columns with StandardScaler before model training.
5	Split the dataset	70/15/15 split into Training (21000), Validation (4500), and Testing (4500) samples.
6	Exploratory Data Analysis	Plotted target distribution, all 23 feature distributions, and feature-vs-target scatter plots.
7	Trained Linear Regression	Baseline model; Test R2 = 0.5918, trained in 0.02s.
8	Tuned Ridge Regression	GridSearchCV over 5 alpha values (cv=5); best alpha = 0.1, Test R2 = 0.5918.
9	Tuned Lasso Regression	GridSearchCV over 5 alpha values (cv=5); best alpha = 10, Test R2 = 0.5946 (best of all 4 models).
10	Tuned Elastic Net Regression	GridSearchCV over alpha x l1_ratio grid; best params alpha=0.01, l1_ratio=0.8, Test R2 = 0.5930.

#	Task	Description
11	Compared coefficients across models	Built a per-feature coefficient table for Linear, Ridge, Lasso, and Elastic Net.
12	Visualized predictions and residuals	Plotted Actual-vs-Predicted and Residual plots for all 4 models.
13	Analyzed overfitting/underfitting	Compared Training MSE vs Validation MSE across Ridge alpha values and across all models (Table 5).

Learning Outcomes

After completing this experiment, I was able to:

- Preprocess a real-world dataset by imputing missing values, encoding categorical features, and standardizing numerical features.
- Split a dataset into training, validation, and test sets for proper model evaluation.
- Implement and train a baseline Linear Regression model for a continuous target variable.
- Apply Ridge, Lasso, and Elastic Net regularization to control model complexity and reduce overfitting.
- Use GridSearchCV with cross-validation to tune hyperparameters such as alpha and l1_ratio.
- Evaluate regression models using MAE, MSE, RMSE, and R2 score on both validation and test sets.
- Visualize predicted-vs-actual values and residuals to assess model fit quality.
- Compare training and validation error to identify overfitting and underfitting behaviour.